--- FILE: anon_hash.R ---

```
#' Clean and Standardize Email Strings
#' @description Prepares email addresses for hashing by removing whitespace
#' and normalizing case.
#' @param x A character vector of email addresses.
#' @return A standardized character vector.
anon_clean_email <- function(x) {
  if (is.null(x)) return(NULL)

  # Standardize: trim whitespace and lowercase
  clean_x <- trimws(tolower(as.character(x)))

  # Ensure NAs are preserved as NA_character_ to avoid hashing the string
"NA"
  clean_x[is.na(x) | x == ""] <- NA_character_

  return(clean_x)
}

#' Generate Readable Research IDs via Salted Hashing
#' @description Creates a 12-character SHA-256 hash from an identifier and
#' a project-specific salt.
#' @param x A character vector of identifiers (e.g., cleaned emails).
#' @return A truncated 12-character hex string vector.
anon_hash <- function(x) {

  # 1. Environment Gate: Retrieve Salt
  # Option A: Salt managed via .Renviron for security and reproducibility
  salt_key <- Sys.getenv("HARMONIZE_SALT")

  # Fail-Fast: Do not allow insecure defaults
  if (salt_key == "") {
    cli::cli_abort(c(
      "x" = "Security Error: No {.var HARMONIZE_SALT} found in
environment.",
      "i" = "Please set a salt in your {.file .Renviron} file: {.code
HARMONIZE_SALT='your_secret_key'}",
      "!" = "Execution halted to prevent insecure de-identification."
    ))
  }

  # 2. Hashing Execution
  # We use SHA-256 via the digest package as it is the current research
standard
  hashed_values <- vapply(x, function(val) {
    if (is.na(val)) return(NA_character_)

    # Concatenate value with salt before hashing
    full_hash <- digest::digest(paste0(val, salt_key), algo = "sha256")

    # 3. Formatting: Truncate to 12 characters for readability as requested
    substr(full_hash, 1, 12)
  }, character(1), USE.NAMES = FALSE)

  return(hashed_values)
}
```

--- FILE: detect_factor_potential.R ---

```
detect_factor_potential <- function(x) {
  # 1. Pre-processing: Isolate non-missing data for logic checks
  # We work on 'x_clean' to calculate metrics, but return original 'x' (or
version of it)
```

```
  x_clean <- x[!is.na(x)]
  n_total <- length(x_clean)

  # Edge case: If vector is empty or all NA, return original class
  if (n_total == 0) return(x)

  # Calculate Key Metrics
  unique_vals <- unique(x_clean)
  n_unique    <- length(unique_vals)

  # ----------------------------------------------------------
  # STRATEGY 1: The 'Likert/Binary' Bypass
  # ----------------------------------------------------------
  # If cardinality is very low, it's likely a factor (e.g., Yes/No, 1-5
scales)
  # regardless of sample size.
  if (n_unique <= 5) {
    return(as.factor(x))
  }

  # ----------------------------------------------------------
  # STRATEGY 2: The 'Pattern' Test (Strict Filters)
  # ----------------------------------------------------------
  # If it failed the bypass, it must pass ALL three rigorous checks
  # to be considered a factor.

  # Condition A: Cardinality Cap
  # Rejects continuous variables (e.g., Age 18-99 has too many levels)
  is_low_cardinality <- n_unique <= 16

  # Condition B: Sample Size Floor
  # We need enough data to distinguish a distribution from noise
  is_large_sample <- n_total >= 20

  # Condition C: Repetition Rule
  # Distinguishes factors (groups) from IDs (unique values).
  # Logic: A true factor should have categories that repeat.
  # We require 20% of the categories to appear at least 3 times.
  freq_counts <- table(x_clean)
  n_repeats   <- sum(freq_counts >= 3)
  rep_ratio   <- n_repeats / n_unique

  is_repetitive <- rep_ratio >= 0.20

  # FINAL DECISION: All strict conditions must be TRUE
  if (is_low_cardinality && is_large_sample && is_repetitive) {
    return(as.factor(x))
  } else {
    return(x)
  }
}


--- FILE: dict_init.R ---
# NOTE: Module A Update (Added 'match_by' logic)
# NOTE: Dependencies: openxlsx, dplyr, purrr, janitor, cli

#' Initialize Data Dictionary
#' @param data_list Named list of dataframes.
#' @param match_by Character. "name" (default) matches by variable name.
"position" matches by column index (risky but good for translations).
#' @param type_prefix Character prefix for target names.
#' @param save_path Path to output .xlsx.
dict_init <- function(data_list, match_by = c("name", "position"),
```

```
                    type_prefix = "var", save_path = "dictionary.xlsx") {

  # 1. SETUP & VALIDATION
  match_by <- match.arg(match_by)

  if (!is.list(data_list) || length(data_list) == 0) {
    cli::cli_abort("Input {.arg data_list} must be a non-empty list of data
frames.")
  }
  if (file.exists(save_path)) {
    cli::cli_alert_warning("File {.file {save_path}} already exists.
Overwriting...")
  }

  # 2. SANITIZATION
  cli::cli_alert_info("Sanitizing input names (janitor::clean_names)...")
  clean_list <- list()
  for (n in names(data_list)) {
    clean_list[[n]] <- janitor::clean_names(data_list[[n]])
  }

  # 3. BUILD WIDE MAP (The Fork in the Road)
  cli::cli_alert_info("Building map using strategy: {.strong {match_by}}")

  if (match_by == "name") {
    # --- STRATEGY A: MATCH BY NAME (Alphabetical) ---
    all_vars_list <- lapply(clean_list, names)
    unique_vars <- sort(unique(unlist(all_vars_list)))

    wide_map <- data.frame(
      target_name = paste0(type_prefix, "_", sprintf("%03d",
seq_along(unique_vars))),
      description = NA_character_,
      stringsAsFactors = FALSE
    )

    for (wave_name in names(clean_list)) {
      col_name <- paste0("orig_", tolower(wave_name))
      is_present <- unique_vars %in% names(clean_list[[wave_name]])
      wide_map[[col_name]] <- ifelse(is_present, unique_vars, NA_character_)
    }

  } else {
    # --- STRATEGY B: MATCH BY POSITION (Structural) ---
    # This aligns Col 1 of W1 with Col 1 of W2.
    # Crucial for multi-language questionnaires.

    max_cols <- max(vapply(clean_list, ncol, integer(1)))

    wide_map <- data.frame(
      target_name = paste0(type_prefix, "_", sprintf("%03d", 1:max_cols)),
      description = NA_character_,
      stringsAsFactors = FALSE
    )

    for (wave_name in names(clean_list)) {
      col_name <- paste0("orig_", tolower(wave_name))
      current_names <- names(clean_list[[wave_name]])

      # Pad with NA if this wave has fewer columns than the max
      length(current_names) <- max_cols
      wide_map[[col_name]] <- current_names
    }
  }
```

```
  # 4. FACTOR MINING
  factor_rows <- list()
  # Start at Row 2 (Row 1 is header)
  current_row_idx <- 2

  for (nm in names(clean_list)) {
    df <- clean_list[[nm]]
    # Detect factors
    vars <- names(df)[vapply(df, function(x)
is.factor(detect_factor_potential(x)), logical(1))]

    if (length(vars) > 0) {
      # Identify the column letter in Wide Map for this wave
      # We use 'wide_map' structure to find the index
      wave_col_name <- paste0("orig_", tolower(nm))
      target_col_idx <- which(names(wide_map) == wave_col_name)
      col_letter <- openxlsx::int2col(target_col_idx) # e.g., "C", "D"

      for (v in vars) {
        vals <- sort(unique(df[[v]]))
        n <- length(vals)

        factor_rows[[length(factor_rows) + 1]] <- data.frame(
          wave = tolower(nm),
          original_variable = v,
          target_name = NA, # The Formula String
          # target_name = formulas, # The Formula String
          original_level = as.character(vals),
          standard_code = seq_along(vals),
          standard_label = as.character(vals),
          stringsAsFactors = FALSE
        )
        current_row_idx <- current_row_idx + n
      }
    }
  }

  factor_map <- dplyr::bind_rows(factor_rows)


  # 5. EXCEL GENERATION
  wb <- openxlsx::createWorkbook()
  header_style <- openxlsx::createStyle(textDecoration = "bold", border =
"Bottom")

  # Instructions
  instr_text <- c(
    "SHEET: Variable_Map_Wide",
    if(match_by == "position") {
    "CAUTION: Variables aligned by COLUMN POSITION. Verify that 'orig_w1'
and 'orig_w2' actually correspond."
  } else {
    "Variables aligned by NAME. Merging different languages may require
manual row moves."
  },
  "",
  "SHEET: Factor_Levels",
    "Column C (target_name) is calculated automatically from target_name in
'Variable_Map_Wide'. Requires Excel 2021/365.",
  "LEGACY FIX: If showing #NAME?, replace formula with VLOOKUP, i.e.
(european standard):",
  "=IFERROR(INDEX('Variable_Map_Wide'!$A:$A; MATCH(B2; 'Variable_Map_Wide'!
$C:$C; 0)); INDEX('Variable_Map_Wide'!$A:$A; MATCH(B2; 'Variable_Map_Wide'!
```

```
$D:$D; 0)))"
  )
  # --- Sheet: Instructions ---
  openxlsx::addWorksheet(wb, "Instructions")
  openxlsx::writeData(wb, "Instructions", x = instr_text, startCol = 1,
startRow = 1)
  # openxlsx::writeData(wb, "Instructions", data.frame(Strategy = match_by,
Note = instr_text))
  # --- Sheet: Variable_Map_Wide ---
  openxlsx::addWorksheet(wb, "Variable_Map_Wide")
  openxlsx::writeData(wb, "Variable_Map_Wide", wide_map)
  openxlsx::addStyle(wb, "Variable_Map_Wide", header_style, rows = 1, cols =
1:ncol(wide_map))
  openxlsx::freezePane(wb, "Variable_Map_Wide", firstCol = TRUE)
  # --- Sheet: Factor Levels ---
  openxlsx::addWorksheet(wb, "Factor_Levels")

    if (nrow(factor_map) > 0) {
      openxlsx::writeDataTable(wb, "Factor_Levels",
                               x = factor_map,
                               tableName = "Table_Factors",
                               tableStyle = "TableStyleMedium2")

    }
  # Embed FORMULAS with =XLOOKUP
  n_meta_rows <- nrow(wide_map) + 1 # Variable map range

  # Create function arguments
  lookup_array_refC <- sprintf("'%s'!$C$2:$C$%d", "Variable_Map_Wide",
n_meta_rows)
  lookup_array_refD <- sprintf("'%s'!$D$2:$D$%d", "Variable_Map_Wide",
n_meta_rows)
  return_array_ref <- sprintf("'%s'!$A$2:$A$%d", "Variable_Map_Wide",
n_meta_rows)
  n_data_rows <- (n_meta_rows-1) * 2
  data_row_indices <- 2:(n_data_rows + 1)

  formula_vec <- paste0(
    "_xlfn.IFNA(",
    "_xlfn.XLOOKUP(",
    "B", data_row_indices, ", ",
    lookup_array_refC, ", ",
    return_array_ref,
    "), ",
    "_xlfn.XLOOKUP(",
    "B", data_row_indices, ", ",
    lookup_array_refD, ", ",
    return_array_ref,
    "))"
  )


  # Write formula into table
  # writeData(wb, sheet = "Factor_Levels", x = "target_name", startCol = 3,
startRow = 1)
  writeFormula(wb, sheet = "Factor_Levels", x = formula_vec, startCol = 3,
startRow = 2)



  # --- Sheet: Change Log ---
  openxlsx::addWorksheet(wb, "Change_Log")
  openxlsx::writeData(wb, "Change_Log", data.frame(timestamp =
as.character(Sys.time()), action = "INIT", match_strategy = match_by))
```

```
  # --- SSave Workbook ---
  openxlsx::saveWorkbook(wb, save_path, overwrite = TRUE)
  cli::cli_alert_success("Dictionary initialized at {.file {save_path}}")

  return(invisible(TRUE))
}



--- FILE: dict_validate.R ---
# NOTE: This is Step 1 (Logic Only). No Roxygen yet.
# NOTE: Dependencies: readxl, cli, stringr, dplyr

dict_validate <- function(path) {

  # 1. FAIL FAST: File Existence
-----------------------------------------
  if (!file.exists(path)) {
    cli::cli_abort("Dictionary file not found: {.file {path}}")
  }

  cli::cli_alert_info("Validating dictionary structure...")

  # 2. SHEET CHECK
------------------------------------------------------------
  sheets <- readxl::excel_sheets(path)
  required <- c("Variable_Map_Wide", "Factor_Levels")
  missing <- setdiff(required, sheets)

  if (length(missing) > 0) {
    cli::cli_abort("Missing critical sheets: {.val {missing}}")
  }

  # 3. VARIABLE MAP VALIDATION
-----------------------------------------------
  # Read as text to prevent eager type coercion
  wide_map <- readxl::read_excel(path, sheet = "Variable_Map_Wide",
col_types = "text")

  # Enforce lowercase headers
  names(wide_map) <- tolower(names(wide_map))

  if (!"target_name" %in% names(wide_map)) {
    cli::cli_abort("Sheet 'Variable_Map_Wide' is missing the {.var
target_name} column.")
  }

  # Check for Duplicates (Critical for Primary Keys)
  # We filter out NAs because sometimes users leave empty rows for spacing
  targets <- wide_map$target_name[!is.na(wide_map$target_name)]

  if (any(duplicated(targets))) {
    dupes <- unique(targets[duplicated(targets)])
    cli::cli_abort(c(
      "Duplicate {.var target_name} detected.",
      "x" = "Concepts must be unique: {.val {dupes}}"
    ))
  }

  # Check for "snake_case" compliance in target_name
  # Regex: starts with letter, only contains letters, numbers, underscores
  bad_names <- targets[!stringr::str_detect(targets, "^[a-z][a-z0-9_]*$")]
  if (length(bad_names) > 0) {
```

```
    cli::cli_alert_danger("The following target_names violate 'snake_case'
rules:")
    cli::cli_ul(head(bad_names, 5))
    cli::cli_abort("All target_names must be lowercase, no spaces, no
special chars.")
  }

  # 4. FACTOR MAP VALIDATION
-----------------------------------------------
  f_map <- readxl::read_excel(path, sheet = "Factor_Levels")
  names(f_map) <- tolower(names(f_map))

  req_cols <- c("target_name", "standard_code", "standard_label")
  missing_f <- setdiff(req_cols, names(f_map))

  if (length(missing_f) > 0) {
    cli::cli_abort("Sheet 'Factor_Levels' missing columns: {.val
{missing_f}}")
  }

  # Ensure standard_code is numeric
  # readxl might have read it as text if there were typos.
  if (!is.numeric(f_map$standard_code)) {
    # Try to coerce
    coerced <- suppressWarnings(as.numeric(f_map$standard_code))
    if (any(is.na(coerced) & !is.na(f_map$standard_code))) {
      cli::cli_abort("Column {.var standard_code} contains non-numeric
values. It must be strictly integers.")
    }
  }

  # Orphan Check: Factor targets that don't exist in Wide Map
  orphans <- setdiff(unique(f_map$target_name),
unique(wide_map$target_name))
  if (length(orphans) > 0) {
    cli::cli_alert_warning("Found {length(orphans)} orphaned factor
definitions (not in Wide Map). They will be ignored.")
  }

  cli::cli_alert_success("Dictionary {.file {basename(path)}} passed
validation.")
  return(invisible(TRUE))
}


--- FILE: harm_add_timepoint.R ---
#' Append a New Longitudinal Timepoint
#'
#' @description Safely joins a new wave of data to an existing master
dataset.
#' It automatically handles variable renaming (suffixing) to prevent
collisions
#' and uses a full join to preserve all participants (attrition +
recruitment).
#'
#' @param master_data A data.frame/tibble representing the current master
dataset (Time 1..N).
#' @param new_data A data.frame/tibble representing the new timepoint (Time
N+1).
#'                 Must be already processed by dict_apply().
#' @param by_id Character. The name of the distinct variable to join by
(e.g., "participant_id").
#'              Must exist in both datasets.
#' @param suffix Character. The suffix to append to new variables (e.g.,
```

```
"_w2", "_t2").
#'               Must start with an underscore/separator to ensure
readability.
#' @param verbose Logical. If TRUE, prints a summary of the join statistics.
#'
#' @return A single wide-format tibble containing both datasets.
#' @export
harm_add_timepoint <- function(master_data, new_data, by_id, suffix, verbose
= TRUE) {

  # 1. FAIL FAST: Input Validation
----------------------------------------
  if (!is.data.frame(master_data) || !is.data.frame(new_data)) {
    cli::cli_abort("Inputs {.arg master_data} and {.arg new_data} must be
data frames.")
  }

  if (!is.character(by_id) || length(by_id) != 1) {
    cli::cli_abort("{.arg by_id} must be a single character string.")
  }

  if (!by_id %in% names(master_data)) {
    cli::cli_abort("ID column {.val {by_id}} not found in {.arg
master_data}.")
  }

  if (!by_id %in% names(new_data)) {
    cli::cli_abort("ID column {.val {by_id}} not found in {.arg new_data}.")
  }

  # Check for duplicate IDs in the NEW data (Critical for 1:1 Joining)
  if (any(duplicated(new_data[[by_id]]))) {
    n_dupes <- sum(duplicated(new_data[[by_id]]))
    cli::cli_abort(c(
      "x" = "Duplicate IDs detected in {.arg new_data}.",
      "i" = "Found {n_dupes} duplicate entries for ID {.val {by_id}}.",
      "!" = "Clean or aggregate the new wave before joining."
    ))
  }

  # 2. SUFFIX LOGIC: Isolate the New Wave
----------------------------------
  # We rename ALL columns in new_data (except the ID)
  # This prevents 'age.x' and 'age.y' messes.

  # Get all columns except the ID
  vars_to_rename <- setdiff(names(new_data), by_id)

  if (length(vars_to_rename) == 0) {
    cli::cli_warn("{.arg new_data} contains only the ID column. Nothing to
join.")
    return(tibble::as_tibble(master_data))
  }

  # Apply suffix
  new_data_renamed <- new_data %>%
    dplyr::rename_with(
      .fn = ~ paste0(., suffix),
      .cols = dplyr::all_of(vars_to_rename)
    )

  # 3. EXECUTE JOIN: Full Join Strategy
-----------------------------------
  # We use full_join to keep:
```

```
  # - People in Master but missing in New (Attrition)
  # - People in New but missing in Master (Recruitment)

  combined_data <- dplyr::full_join(master_data, new_data_renamed, by =
by_id)

  # 4. AUDIT: Human-in-the-Loop Reporting
-----------------------------------
  if (verbose) {
    # Calculate overlap stats
    ids_master <- master_data[[by_id]]
    ids_new    <- new_data[[by_id]]

    n_match    <- length(intersect(ids_master, ids_new))
    n_attrit   <- length(setdiff(ids_master, ids_new)) # Only in Master
    n_recruit  <- length(setdiff(ids_new, ids_master)) # Only in New

    cli::cli_h2("Longitudinal Join Audit")
    cli::cli_ul(c(
      "Matched (Both Waves): {.val {n_match}}",
      "Attrition (Master Only): {.val {n_attrit}}",
      "Recruitment (New Only): {.val {n_recruit}}",
      "Total Participants: {.val {nrow(combined_data)}}"
    ))

    if (n_recruit > 0) {
      cli::cli_alert_info("New participants added. Check if these are valid
new recruits or ID typos.")
    }
  }

  return(tibble::as_tibble(combined_data))
}


--- FILE: harm_bind_waves.R ---
#' Apply Dictionary Mapping and Standardize Categorical Values
#' @description Renames variables based on the Wide Map and converts factor
#' labels to numeric strings using the Factor Levels.
#' @param data A data.frame to be standardized.
#' @param wb_path Path to the Excel dictionary (.xlsx).
#' @param wave_name The specific wave identifier matching the dictionary
columns (e.g., "w24").
#' @return A standardized tibble.
dict_apply <- function(data, wb_path, wave_name) {

  # 1. Initialization and Sanitization
  # Input is immediately passed through janitor::clean_names
  data_clean <- janitor::clean_names(data)
  wave_id <- tolower(wave_name)

  # Load mapping sheets using openxlsx for compatibility with existing
codebase
  w_map <- openxlsx::read.xlsx(wb_path, sheet = "Variable_Map_Wide")
  l_map <- openxlsx::read.xlsx(wb_path, sheet = "Factor_Levels")

  # Force lowercase headers to prevent case-sensitivity crashes
  colnames(w_map) <- tolower(colnames(w_map))
  colnames(l_map) <- tolower(colnames(l_map))

  # Identify the specific column for this wave (e.g., 'orig_w24')
  wave_col <- paste0("orig_", wave_id)
  if (!wave_col %in% colnames(w_map)) {
    cli::cli_abort("Wave column {.val {wave_col}} not found in
```

```
Variable_Map_Wide.")
  }

  # 2. Variable Renaming and Subsetting
  # Extract mapping for variables present in this wave
  map_sub <- w_map[!is.na(w_map[[wave_col]]), c("target_name", wave_col)]

  # Only keep variables that are explicitly mapped in the dictionary
  valid_cols <- map_sub[[wave_col]] %in% colnames(data_clean)
  map_sub <- map_sub[valid_cols, ]

  if (nrow(map_sub) == 0) {
    cli::cli_warn("No variables in wave {.val {wave_name}} matched the
dictionary.")
    return(tibble::as_tibble(data_clean))
  }

  # Subset and rename in one step
  data_std <- data_clean[, map_sub[[wave_col]], drop = FALSE]
  colnames(data_std) <- map_sub$target_name

  # 3. Categorical Transformation (Factor Re-leveling)
  # Filter the factor map for this specific wave
  wave_factors <- l_map[l_map$wave == wave_id, ]

  if (nrow(wave_factors) > 0) {
    for (orig_var in unique(wave_factors$original_variable)) {

      # Find the target_name associated with this original variable
      target <- map_sub$target_name[map_sub[[wave_col]] == orig_var]

      if (length(target) > 0 && target %in% colnames(data_std)) {
        lev_map <- wave_factors[wave_factors$original_variable ==
orig_var, ]

        # Mapping: Original Label -> Standard Code (as character string)
        # This allows cross-language binding by unifying labels to numbers
        lookup <- stats::setNames(as.character(lev_map$standard_code),
                                  as.character(lev_map$original_level))

        # Apply the mapping while maintaining the factor class
        # We use levels defined in the dictionary to ensure a universal
baseline
        data_std[[target]] <- factor(
          lookup[as.character(data_std[[target]])],
          levels = unique(as.character(lev_map$standard_code))
        )
      }
    }
  }

  return(tibble::as_tibble(data_std))
}

#' Robustly Bind Harmonized Waves
#' @description Combines a list of standardized dataframes, handling
potential
#' type mismatches safely.
#' @param data_list A named list of dataframes already processed by
dict_apply.
#' @return A single harmonized tibble.
harm_bind_waves <- function(data_list) {

  if (!is.list(data_list) || length(data_list) == 0) {
```

```
    cli::cli_abort("Input must be a non-empty list of dataframes.")
  }

  # Safety Net: Use the resolve_types logic for any lingering
inconsistencies
  # bind_rows handles the heavy lifting, but we add a 'source_wave' ID
  combined <- dplyr::bind_rows(data_list, .id = "source_wave")

  # Ensure all variables created are snake_case
  colnames(combined) <- tolower(colnames(combined))

  return(tibble::as_tibble(combined))
}


--- FILE: id_audit.R ---
#' Audit IDs for Longitudinal Typos
#' @description Identifies potential typos in IDs/emails intended for
longitudinal
#' column-binding. Compares IDs across different time points.
#' @param data_list A list of dataframes containing the ID column.
#' @param id_col Character. The name of the ID column (e.g., "pk_hash" or
"email").
#' @param wb_path Character. Path to the Excel dictionary.
#' @param max_dist Numeric. Maximum Levenshtein distance for a typo flag.
check_id_audit <- function(data_list, id_col, wb_path, max_dist = 1) {

  # 1. Collect unique IDs across all longitudinal time points
  all_ids <- unique(unlist(lapply(data_list, function(df)
as.character(df[[id_col]]))))
  all_ids <- all_ids[!is.na(all_ids) & all_ids != ""]

  if (length(all_ids) < 2) return(invisible(NULL))

  # 2. String Distance Matrix (Levenshtein)
  dist_matrix <- stringdist::stringdistmatrix(all_ids, all_ids, method =
"lv")

  # Find pairs within the user-adjustable threshold
  mismatch_indices <- which(dist_matrix > 0 & dist_matrix <= max_dist &
upper.tri(dist_matrix), arr.ind = TRUE)

  if (nrow(mismatch_indices) > 0) {
    report <- data.frame(
      id_a = all_ids[mismatch_indices[, 1]],
      id_b = all_ids[mismatch_indices[, 2]],
      distance = dist_matrix[mismatch_indices],
      action_required = "Check if these are the same participant",
      stringsAsFactors = FALSE
    )

    # 3. Append to Excel Review Sheet
    wb <- openxlsx::loadWorkbook(wb_path)
    sheet_name <- "Review_IDs"

    if (sheet_name %in% names(wb)) openxlsx::removeWorksheet(wb, sheet_name)
    openxlsx::addWorksheet(wb, sheet_name)
    openxlsx::writeData(wb, sheet_name, report)

    # Styling for Human-in-the-Loop
    warn_style <- openxlsx::createStyle(fontColour = "#9C0006", bgFill =
"#FFC7CE")
    openxlsx::conditionalFormatting(wb, sheet_name, cols = 1:ncol(report),
                                    rows = 2:(nrow(report)+1),
```

```
                                    rule = '!= ""', type = "expression",
style = warn_style)

    openxlsx::saveWorkbook(wb, wb_path, overwrite = TRUE)
    cli::cli_alert_info("ID Audit complete: {.val {nrow(report)}} potential
typos added to {.file {sheet_name}}.")
  }
}

#' Final Integrity Check for Tidy Harmonized Data
#' @description Verifies uniqueness of records and adherence to categorical
data contracts.
#' @param data The final bound tibble.
#' @param id_col Character. The primary key column (e.g., "pk_hash").
#' @param time_col Character. The column identifying the time point.
#' @param verbose Logical. If TRUE, prints a success message. Defaults to
TRUE.
check_integrity <- function(data, id_col, time_col = "source_wave", verbose
= TRUE) {

  # 1. Tidy Data Check: Unique ID + Time
  duplicates <- duplicated(data[, c(id_col, time_col)])
  if (any(duplicates)) {
    n_dupes <- sum(duplicates)
    cli::cli_abort(c(
      "x" = "Integrity Failure: Non-tidy data detected.",
      "i" = "{.val {n_dupes}} duplicate combinations of {.var {id_col}} and
{.var {time_col}} found.",
      "!" = "Each participant must have exactly one row per time point."
    ))
  }

  # 2. Type Contract Check
  # Ensure all columns that should be categorical are factors with
character-numeric labels
  is_factor_check <- vapply(data, is.factor, logical(1))

  # Check if factor levels are numeric strings (Project Blueprint
Requirement)
  for (col in names(data)[is_factor_check]) {
    lvls <- levels(data[[col]])
    if (!all(grepl("^[0-9]+$", lvls))) {
      cli::cli_warn("Variable {.var {col}} is a factor but contains non-
numeric labels.")
    }
  }

  # 3. Conditional Success Message
  if (verbose) {
    cli::cli_alert_success("Integrity Check Passed: Data is tidy and
fulfills contract requirements.")
  }
  return(invisible(TRUE))
}
```